hashlock.

Security Audit

Energy Web (Worker Node Pallet)

November 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Energy Web team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

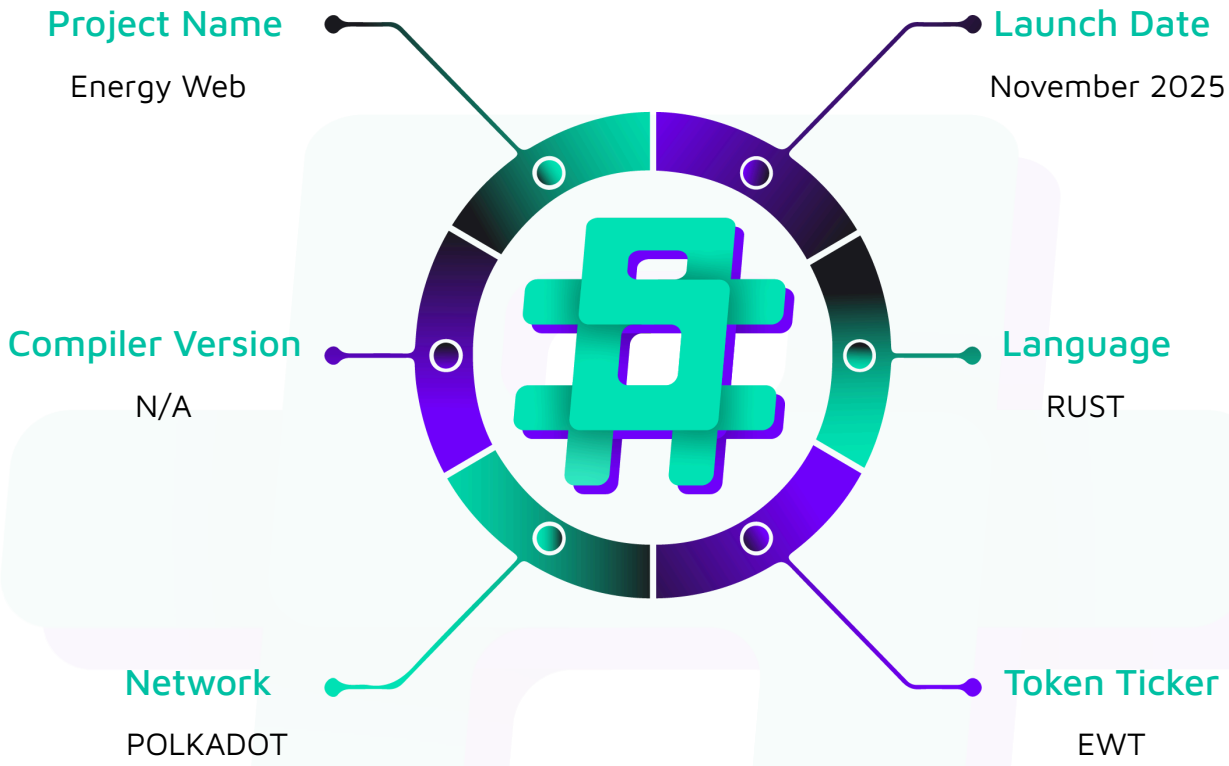
Project Context

Energy Web is a global, open-source nonprofit focused on accelerating the clean energy transition through decentralized digital infrastructure. Launched in 2017, the organization stewarded the Energy Web Chain (EWC), an enterprise-focused Proof-of-Authority blockchain. Energy Web is now transitioning to it's flagship network: Energy Web X (EWX), a Substrate-based Polkadot parachain.

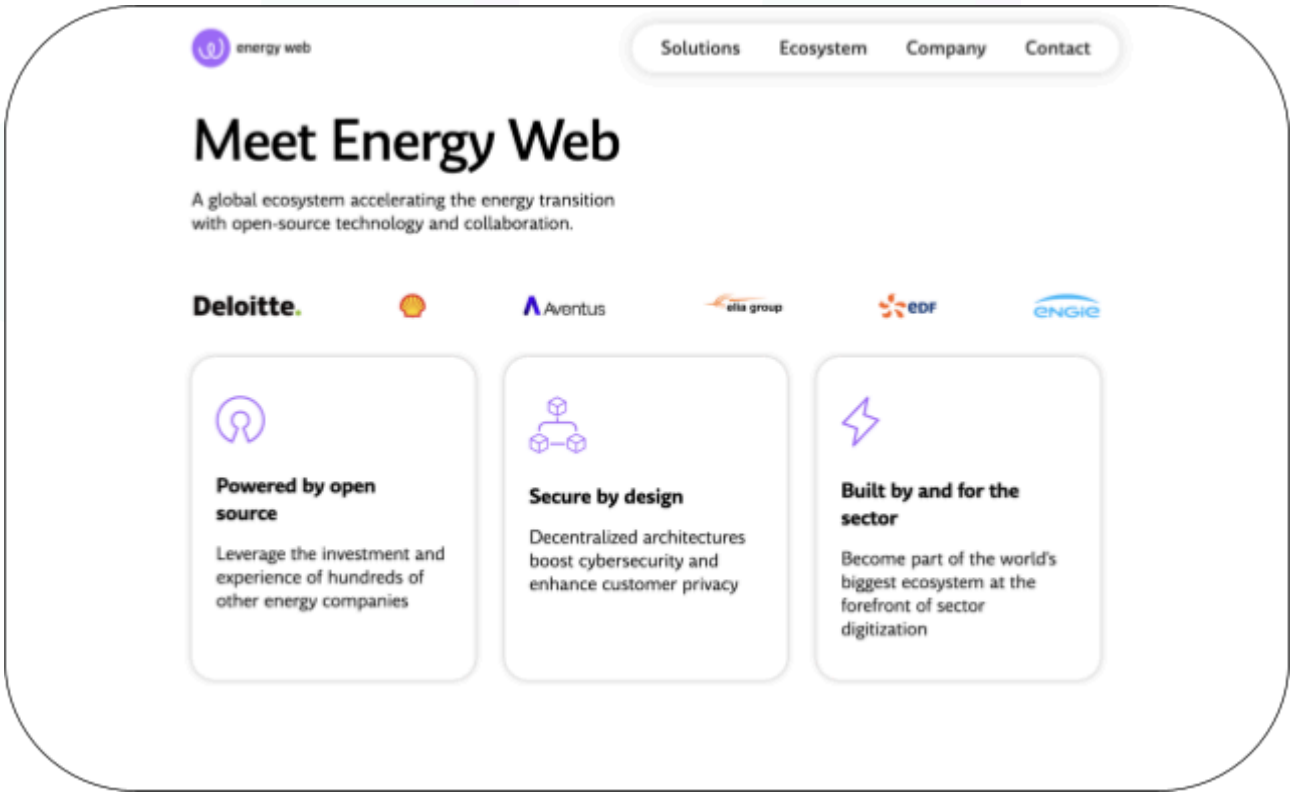
EWX introduces a permissionless Proof-of-Stake consensus model, enabling broad validator and delegator participation while unlocking staking rewards for participants and supporting a robust on-chain economy. To expand liquidity and cross-chain functionality, EWX integrates natively with Polkadot Asset Hub, enabling seamless registration, transfer, and utilization of foreign assets within the Energy Web ecosystem while maintaining compatibility with Ethereum and the legacy EWC network.

Project Name: Energy Web
Project Type: DeFi, Token, Bridge
Website: https://www.energyweb.org/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the Rust code within the Energy Web project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Energy Web Protocol Smart Contracts**
Platform	**Polkadot / Rust**
Audit Date	**November, 2025**
Contract 1	data_structs.rs
Contract 2	lib.rs
Contract 3	mod.rs
Contract 4	v8.rs
Contract 5	rewards.rs
Contract 6	slashing.rs
Contract 7	voting.rs
Audited GitHub Commit Hash	2165599ccd096232b6e63512b61dedd1a96e14e2
Fix Review GitHub Commit Hash	25a3c3353cda1170e12185e985a8af01f2d629d6

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Hashlocked"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

3 Medium severity vulnerabilities

2 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Ewx-worker-solution pallet: - Manages staking for solution groups, - Tracks voting rounds and consensus, - Calculates rewards per period automatically, - Handles operator subscriptions and withdrawals, - Supports both native and asset staking.	**Pallet achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Energy Web project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Energy Web project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] pallet#stake_manager - Asset stake release can under-release while stake is fully removed from records

Vulnerability Details

For asset-stake groups, `stake_manager::unstake` does:

```
match stake_currency {
        StakeCurrency::Asset => {
             // TODO: we need to change this to exclude the possibility of failed unstake because of
            // failed conversion
            let assets = SolutionWorker::<T>::balance_to_stake(unstake_amount)?;
            T::AssetsHolder::release(
                T::StakeAssetId::get(),
                &T::StakeAssetHoldReason::get(),
                &account,
                assets,
                Precision::BestEffort,
            )
            .map_err(|e| {
                  log::error!("Failed to release {:?} from {:?}: {:?}", assets, account, e);
                Error::<T>::FailedToReleaseStake
            })?;
        },
        StakeCurrency::Native => {
                                                    let     unstake_balance     = SolutionWorker::<T>::balance_to_currency(unstake_amount);
            T::DepositCurrency::unreserve(account, unstake_balance);
```

```
        },
    }


    Ok(unstake_amount)
```

and always returns `Ok(unstake_amount)` on success, regardless of how many assets were actually released.

With `Precision::BestEffort`, the release call may return `Ok(actual_released)` where `actual_released < assets` if there aren't enough held assets. The pallet ignores the returned amount and then:

- Updates `StakeRecord` to remove `unstake_amount` worth of stake, and
- Updates group reward metadata as if that stake is gone.

So you can end up with a stake record of zero (no more stake), but some stake assets are still held under `StakeAssetHoldReason`, which are now unreachable via pallet `APIs`.

This can realistically happen if there is any future logic that slashes or otherwise reduces the held stake asset outside this function, or rounding or converter changes mean `balance_to_stake(unstake_amount)` now asks for slightly more assets than are actually held.

Impact

Residual staked assets can remain locked under the hold while the pallet believes the stake is fully withdrawn.

Recommendation

Use `Precision::Exact` for stake release and treat any error as `FailedToReleaseStake`, leaving the `StakeRecord` unchanged so the unsubscribe can be retried. Alternatively, compare the returned amount from `release` to the requested `assets` and treat any shortfall as an error - do not decrement the stake record in that case.

Status

Resolved

Medium

[M-01] pallet#lib.rs - Asset rewards can become unclaimable if converter behavior changes after registration

Vulnerability Details

For asset-denominated reward groups (`RewardsCurrency::Asset(asset_id)`), at registration, the deposit fee is computed as a `Balance` (`EWT` units) and converted to an asset amount via `balance_to_assets(rewards, asset_id)` before being put on hold.

During reward calculation, per-period rewards are computed as `Balance` and converted into `BalanceOf<T>` (native) via `balance_to_currency` for storage in `EarnedRewards` and `SolutionGroupCalculatedRewards`.

At claim time, the accumulated `reward: BalanceOf<T>` is converted back to assets with:

```
RewardsCurrency::Asset(asset_id) => {
                            let hold_reason = T::RewardsAssetHoldReason::get();
                            // TODO: we need to change this to exclude the possibility of
a failed claim
                            // because of failed conversion
                            let assets_to_claim = Self::balance_to_assets(
                                Self::currency_to_balance(reward),
                                asset_id.clone(),
                            )?;
                            T::AssetsHolder::transfer_on_hold(
                                asset_id,
                                &hold_reason,
                                &owner,
                                &sender,
                                assets_to_claim,
                                Precision::Exact,
                                Restriction::Free,
                                Fortitude::Polite,
```

```
                        )
                        .map_err(|_| {
                            log::error!("Failed to release {:?}", assets_to_claim);
                            Error::<T>::RewardsDepositInsufficient
                        })?;
                },
```

If `AssetBalanceConverter` behavior changes between registration and claim, for example due to the runtime upgrade, decimal handling improvements or config parameter adjustments, the `EWT-equivalent reward` may convert into an asset amount that is larger than what was originally held. `transfer_on_hold` will then fail with `RewardsDepositInsufficient`, and the entire `claim_rewards` call reverts, leaving both the rewards and underlying reward deposit locked.

Impact

Legitimately earned rewards may become impossible to claim for asset-reward groups.

Recommendation

Treat `AssetBalanceConverter` as immutable and strictly `1:1` for the configured reward assets, as the comment suggests ("... to the same amount of the native token"), and document that requirement in runtime configuration.

Status

Resolved

[M-02] pallet#lib.rs - Shared on-hold reward pool per owner allows one group to consume the deposit intended for another

Vulnerability Details

Asset reward deposits are held under a single `(owner, asset_id, RewardsAssetHoldReason)`:

```
RewardsCurrency::Asset(asset_id) => {
                    let assets = Self::balance_to_assets(net_fee, asset_id.clone())?;
                    T::AssetsHolder::hold(
                        asset_id.clone(),
                        &T::RewardsAssetHoldReason::get(),
                        &sender,
                        assets,
                    )
                    .map_err(|e| {
                        log::error!("{:?}", e);
                        Error::<T>::InsufficientFunds
                    })?;
                },
```

When paying out rewards, `claim_rewards` calls `transfer_on_hold` using only the asset ID, hold reason, owner, and amount, with no per-group partitioning at the holder level.

Per-group limits are enforced only in `SolutionGroupCalculatedRewards` (`EWT-equivalent` deposited vs. claimed), but the underlying on-hold balance is a "single" bucket per owner, asset, and reason.

If, for whatever reason, the `SolutionGroupCalculatedRewards0` for one group understates what was actually deposited, that group may be allowed to claim an overly large reward in `EWT` terms. As long as the global on-hold asset balance is sufficient, because other groups' deposits share the same pool, `transfer_on_hold` will succeed - effectively consuming collateral intended for other groups.

Impact

Mis-accounted or mis-migrated groups can drain reward collateral from other groups owned by the same registrar.

Recommendation

Introduce per-group isolation at the holder level, for example, by encoding the group namespace into the hold reason, or using separate reason IDs per group.

Status

Resolved

[M-03] pallet#lib.rs - The claim_rewards drops namespaces with missing SolutionsGroups entry

Vulnerability Details

The new `claim_rewards` builds a list of groups and rewards like this:

```
let selected_namespaces: Vec<(EntityNamespace, BalanceOf<T>)> = group_namespaces
                .into_iter()
                .map(|namespace| {
                    let reward = <EarnedRewards<T>>::take(&sender, &namespace);
                    (namespace, reward.0.saturating_add(reward.1))
                })
                .collect();
```

If a namespace has `EarnedRewards` but no `SolutionsGroups` entry, for example, after deregistration while rewards are still outstanding, it is silently dropped from `selected_namespaces`, and thus never paid out. The older version did not require `SolutionsGroups::<T>::get` to succeed - it only needed `SolutionsGroupsInventory` to find the owner.

The pallet does track deregistered groups with `DeregisteredGroupsWithRewards`, but this is only used to decide when all rewards are claimed, and the group can be fully removed, not to allow claims without `SolutionsGroups`.

Impact

Rewards for deregistered or migrated groups can become permanently unclaimable.

Recommendation

When `SolutionsGroups::<T>::get(&namespace)` returns `None` but `DeregisteredGroups WithRewards::<T>` indicates outstanding rewards, fall back to a stored `RewardsCurrency`.

Status

Resolved

Low

[L-01] pallet#lib.rs - The RewardsClaimed.total_rewards_claimed event obscures asset vs native breakdown

Description

Currently, the `RewardsClaimed` remains:

```
/// Signals the claiming of rewards

        RewardsClaimed { operator: T::AccountId, total_rewards_claimed: BalanceOf<T> },
```

and `total_rewards_claimed` is computed as the sum of per-group `reward` values, which are stored in `BalanceOf<T>` even for asset groups.

For asset groups, the `reward` value is the `EWT`-equivalent used in accounting, not the actual asset amount transferred. There is no event that exposes "you received X `EWT` and `Y` `stEWT`".

Off-chain monitoring can't verify asset payouts or detect converter misconfigurations from events alone.

Recommendation

Extend `RewardsClaimed` with a per-currency breakdown.

Status

Acknowledged

[L-02] pallet#lib.rs - Slashing configuration added but not yet wired to asset staking

Description

The `SolutionGroup` now has `stake_currency` and `slashing_config` fields. Asset staking uses `AssetsHolder` and `AssetBalanceConverter`, converting the balance of a `StakeAssetId` to the same amount of the native token. The new slashing configuration code (and related structures) currently operates only on logical stake (`StakeRecord`) and does not call into `AssetsHolder` at all.

There is no obvious, unified slash stake `API` that atomically:

- Reduces the logical stake used for reward calculations, and
- Burns or transfers the underlying staked assets.

This gap doesn't break current behavior, but it becomes dangerous as soon as someone wires up slashing using only one of these dimensions.

Recommendation

Before enabling slashing for asset-stake groups, define and implement a single authoritative slashing function.

Status

Acknowledged

Centralisation

The Energy Web project is moving toward full decentralization by having many independent validators make all key decisions instead of a single team. A temporary admin role is only in place during upgrades, after which governance will be fully community-driven.

Conclusion

After Hashlock's analysis, the Energy Web project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au